

## Foundation of Blockchain Technology (IS501) Pre-Reading Material – Lecture 17

### Solidity Data Modifiers

**Estimated Reading Time:** 15–20 minutes

### Why This Topic Matters

In a smart contract, not every variable or function should be accessible to everyone. **Visibility modifiers** determine who can access a variable or invoke a function.

They are important for **security, access control, and protecting contract logic**.

### Learning Outcomes

After completing this pre-reading, you should be able to:

- Explain the concept of **visibility** in Solidity.
- Identify the four main visibility modifiers.
- Differentiate between public, external, internal, and private.
- Understand how visibility affects functions and variables.

### 1. What Is Visibility?

**Visibility** defines who can access a variable or call a function in a Solidity smart contract.

Think of a smart contract as a building:

- **Variables** are like buckets that contain data.
- **Functions** are like hands that perform actions or access the buckets.
- **Visibility modifiers** determine who is allowed to enter or access them.

Visibility can therefore control whether something is accessible **inside the contract, outside the contract, or both**.

### 2. Why Is Visibility Important?

Visibility modifiers help to:

- Control access to contract functions and variables.

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



- Protect internal contract logic.
- Prevent unintended access.
- Define what external users and applications can interact with.

### 3. Four Main Visibility Modifiers

| Modifier | Accessibility               | Common Use                                              |
|----------|-----------------------------|---------------------------------------------------------|
| public   | Internal + External         | Functions or variables intended to be openly accessible |
| external | Outside the contract        | Functions mainly intended for external calls            |
| internal | Current + derived contracts | Shared contract logic and inheritance                   |
| private  | Current contract only       | Restricted functions and variables                      |

### 4. Public

The **public** modifier allows access from both **inside and outside the contract**.

For functions, external users or applications can call the function.

Example:

```
function getBalance() public view returns(uint) {
    // ...
}
```

Public variables can also be accessed externally. Solidity automatically provides a **getter function** for public state variables.

Example:

```
uint public totalSupply;
```

This allows the value of totalSupply to be read externally.

### Think Before the Lecture

1. Why should a smart contract restrict access to some functions?
2. What is the difference between **public** and **private** visibility?
3. Which visibility modifier would you use for functionality intended to be accessed by an external application?
4. Why is visibility important for smart-contract security?
5. If a variable is declared public, who can read it?